

1ª AVALIAÇÃO

Utilizando o processo de abstração, implemente uma classe em Python que represente um **boleto bancário**. Identifique atributos mutáveis e imutáveis, implemente um construtor da classe e métodos para manipulação dos atributos mutáveis. Faça todas as validações necessárias. Utilize encapsulamento e decoradores de leitura/escrita. Crie objetos para testar os métodos implementados através de testes unitários com Pytest (mínimo de uma função de teste para cada método implementado incluindo o construtor).

Especificações:

1. Atributos:

- Número do boleto (gerado automaticamente e único)
 - Nome do pagador
 - Valor original
 - Data de emissão (registrada automaticamente na criação do boleto) //Não pode ser uma data menor que a data atual
 - Data de vencimento (definida no momento da criação)
 - Data de pagamento (registrada quando pago) // Inicialmente: None
 - Status do boleto: "em aberto", "pago", "vencido", "cancelado" (Inicialmente: "em aberto")
 - Valor final com juros e/ou desconto (inicialmente: 0,00)
-

2. Métodos sugeridos:

- Um método para **registrar o pagamento**:
 - Define a data de pagamento
 - Calcula o valor final (com desconto ou juros)
 - Altera o status para "pago" ou "vencido" conforme a data
- Um método para **consultar o valor atualizado**:
 - Considera:
 - **Desconto de 5%** se pago até 3 dias antes do vencimento

- **Multa de 2% + juros de 0,03% ao dia** se pago após o vencimento
 - Um método para **cancelar** o boleto, que altera o status do boleto para “cancelado”. Não pode cancelar se o boleto já foi pago.
-

✓ Regras de Validação:

- O número do boleto deve ser único
- O valor original deve ser positivo
- A data de vencimento deve ser posterior à emissão
- A data de pagamento não pode ser anterior à emissão
- O status só pode ser "pago" se o pagamento for registrado
- O status deve mudar para "vencido" se a data atual ultrapassar o vencimento e o boleto ainda não tiver sido pago
- Boleto com o status “pago” não pode ser pago novamente.

🧪 Teste (pytest):

- Crie pelo menos **uma função de teste** para cada método implementado incluindo o construtor. Simular diferentes datas de vencimento e pagamentos antecipados, pontuais e atrasados.
- Mostre os métodos implementados e suas validações funcionando corretamente.